

Обучение скрытых слоёв S–A–R перцептрона без вычисления градиентов

С. Яковлев mg.sc.comp e-mail: tac1402@gmail.com

Аннотация. Классический перцептрон Розенблатта с архитектурой S–A–R исторически не имел устойчивого алгоритма обучения многослойных структур. В результате в современном машинном обучении доминирует метод обратного распространения ошибки (backpropagation), основанный на градиентном спуске. Несмотря на успехи, этот подход имеет фундаментальные ограничения: необходимость вычисления производных нелинейных функций и высокая вычислительная сложность. В данной работе показано, что при интерпретации работы нейросети через алгоритм ID3 (Rule Extraction) скрытый слой автоматически формирует чистые окрестности в смысле кластерного анализа — признаки группируются по классам ещё до завершения обучения. На основе этого наблюдения автором предложен новый стохастический алгоритм обучения, восходящий к идеям Розенблатта, но принципиально расширяющий их: он позволяет обучать скрытые слои перцептрона без вычисления градиентов. Таким образом, впервые решается классическая проблема обучения архитектуры S–A–R без градиентных методов. Это открывает путь к созданию принципиально новых алгоритмов обучения нейросетей с более простой и интерпретируемой динамикой.

Training Hidden Layers of S–A–R Perceptrons without Gradient Computation

Abstract. The classical Rosenblatt perceptron with the S–A–R architecture has historically lacked a stable algorithm for training multilayer structures. As a result, modern machine learning is dominated by the error backpropagation method (backpropagation) based on gradient descent. Despite its success, this approach has fundamental limitations: the need to compute derivatives of nonlinear functions and high computational complexity. In this work, it is shown that when interpreting the operation of a neural network using the ID3 algorithm (Rule Extraction), the hidden layer automatically forms pure neighborhoods in the sense of cluster analysis—features are grouped into classes even before training is complete. Based on this observation, the author proposes a new stochastic training algorithm, rooted in Rosenblatt’s original ideas but fundamentally extending them: it enables the training of hidden layers of the perceptron without computing gradients. Thus, for the first time, the classical problem of training the S–A–R architecture without gradient-based methods is solved. This opens the way to the development of fundamentally new neural network training algorithms with simpler and more interpretable dynamics.

Введение. Перцептрон, как искусственная нейронная сеть, разрабатывался Ф.Розенблаттом в 1957-61 года [1, 2]. Только 1969 г. М. Минский и С. Паперт публикуют свой анализ ограничений перцептрона [3]. К сожалению, в 1971 г. Розенблатт погибает, так и не ответив на их критику. А DARPA прекращает финансирование, о чем упоминается в отчете 1989 года [4], в связи с желанием возобновить финансирование работ в области коннективизма. В центре, внимания тогда рассматривалась работа о новом подходе к обучению нейросетей – backpropagation [5]. Благодаря, упрощенному изложению в отчете DARPA и борьбе за финансирование возникают множественные неточности и недоразумения, которые отмечали некоторые ученые как раньше [6], так и по сей день [7]. Показывают успешность применения классического перцептрона Розенблатта с архитектурой S-A-R на задаче MNIST [8]. Но в целом, о перцептроне Розенблатта в научном сообществе забывают, и понимание о нем вырождается в однослойный перцептрон, которым он никогда не был. Поэтому следует напомнить, что хотя Розенблатт и предлагал свой “метод коррекции с обратной передачей сигнала ошибки”, он был стохастическим и имел плохую сходимость. К сожалению, последующие ученые недооценили роль случайности, используемую в работе перцептрона. Автором ранее уже были предложены практические уточнения [9] достаточных условий для того, чтобы перцептрон был в состоянии сформировать пространство, удовлетворяющее гипотезе компактности. Но на практике так и оставалось не ясным как обучать скрытый слой перцептрона, без вычисления градиентов, в отличие от того, как это делается в алгоритме backpropagation. Конечно, автор понимает, что успех метода backpropagation и его результаты может затмевать любые альтернативные способы обучения нейросетей. И мы их покажем в сравнении на простой задаче четности. Но учитывая, как много времени и усилий различных ученых прямо или косвенно были посвящены различным аспектам работы нейросетей, в основе которых находится алгоритм backpropagation, не стоит ждать от альтернативных методов сразу потрясающих результатов. Дело в том, что прямо сравнивать MLP+backpropagation vs. Perceptron сложно, т.к. они принципиально по-разному решают задачи классификации. И мы постепенно, пройдем от наивной реализации на C#, через анализ и интерпретацию работы перцептрона, и только затем сделаем отдельные оптимизации.

1. Так экспоненциально или линейно?

Часто можно услышать вывод, что перцептрон Розенблатта требует экспоненциально больше нейронов, чем алгоритм backpropagation. В этом разделе покажем, что это не соответствует истине. Экспоненциальность относится к росту состояний в скрытом слое, а не к числу нейронов. Само же число нейронов, требуемое для решения все более сложной задачи, растет линейно. Здесь и далее, для демонстрации характеристик нейросетей мы будем использовать задачу четности.

Задача четность. Задача чётности заключается в том, что на вход подаются бинарные векторы фиксированной длины, а выход определяется как 1, если число единиц во входе нечётное, и 0 — если чётное. Таким образом, для каждого входного вектора требуется учесть *глобальное* свойство всей комбинации, а не локальные признаки отдельных разрядов.

Множества «чётных» и «нечётных» векторов образуют в пространстве входов структуру, где элементы разных классов чередуются при любом изменении одного из битов. Это означает, что границы классов нельзя провести локально — для правильного ответа необходимо учитывать комбинацию всех входных разрядов одновременно.

Визуально, эту задачу можно представить себе, как шахматную доску, где в середине каждой клетки находится точка соответствующего цвета, и задача перцептрона построить такие линии, чтобы отделить все белые точки в один класс, а все черные в другой. Это будет задача четности с 8 битами. Но далее мы будем решать задачу четности с 10, 12, 14, 16 и более битами.

Экспоненциальность относится как к перцептрону, так и к MLP+backpropagation. Если у нас 16 входов, то они образуют $2^{16} = 65536$ возможных примеров, то в скрытом слое, нам понадобится экспоненциальное число возможных состояний, а именно, не менее 2^{32} , что обеспечивается 32 нейронами. И действительно, MLP+backpropagation не сходится, если для разделения использовать меньшее число нейронов.

Что же касается, перцептрона максимально требуется 65536 нейронов, что равно общему числу примеров. Это и было доказано Розенблаттом. Но кроме того, мы знаем, что существует метод CC4 [8], который при таком количестве нейронов может выставить весовые связи S-A элементов по простому правилу. Таким образом, обеспечив схождение за 1 итерацию. Почему же, тогда нужно правило Хебба, используемое в обучении перцептрона? Дело в том, что 65536 нейронов это теоретический максимум, который на практике существенно меньший. А вот на сколько он меньше исключительно зависит от решаемой задачи. Но не одна даже самая теоретически сложная задача, как например обучения коду Грея никогда не потребует теоретический максимума.

Задача код Грея. Код Грея — это способ пронумеровать все двоичные комбинации так, чтобы соседние числа отличались ровно на 1 бит.

Почему эта задача интересна? Дело в том, что уже две задачи четность и инверсия — это два полюса сложности для ИНС:

- *Чётность*: это глобальная задача, чтобы найти решение, нужно знать контекст всех битов сразу;
- *Инверсия*: это максимально локальная задача, в которой каждый вход соответствует *уникальному* выходу. По отдельности это легко решается, но требует столько же выходов (представляющих классы), сколько и входов.

Чтобы вычислить код Грея необходимо одновременно знать контекст всех битов, и отразить решение на уникальные выходы, причем выходов столько же сколько и входов. Таким образом, при нахождении кода Грея совмещаются особенности задачи четность и инверсии в одной.

Теоретически эта задача должна требовать равенства нейронов числу всех примеров ($A_{\text{Count}} = H_{\text{Count}}$), причем возможность распараллелить вычисления так же, как в задаче четность минимальна, но она есть. На практике, здесь понадобилось около 5000 A – элементов, на полном 16 битном поле стимулов.

Мы можем, так же сказать, что не имеет теоретического смысла, рассматривать другие практические примеры, т.к. любая практическая задача будет или тривиально проще, или комбинацией этих задач.

Проведенные тесты для задачи четности результаты представлены в таблице 1.

Таблица 1.

Обучение классического перцептрона Розенблатта

Кол-во бит	Кол-во возможных комбинаций	Число нейронов в скрытом слое	Итераций	Время	Требуемая память
8	256	150*	17	< 1 сек.	15 Мб
10	1024	300*	26-65	< 1 сек.	16 Мб
12	4096	600*	314	< 2 сек.	17 Мб
14	16384	1400	150	< 2 сек.	25 Мб
16	65536	3000	383	60 сек.	78 Мб
17	131072	4500	821	6 мин.	178 Мб
18	262144	6000	574	10 мин.	401 Мб
19	524288	9000	367	20 мин.	1, 0 Гб
20	1048576	12000	529	69 мин.	2,4 Гб

* - при числе нейронов < 1000 высока вероятность отсутствия решения, поэтому указано ориентировочное число для практического решения.

В таблицах даны условные цифры, чтобы можно было понять порядок величин. Хотя автор и пытался задать как можно меньшее число нейронов и при этом минимизируя общее время нахождения решения, понятно, что это число может быть немногим меньше, если подождать существенно дольше, или найти лучший random seed, или наоборот, дать чуть больше нейронов, чтобы схождение произошло быстрее. Эти цифры надо воспринимать как вырожденный случай, который показывает, на каком минимальном числе нейронов практически возможно произвести обучение за разумное время.

Из таблицы 1. видим, что требуемое число нейронов в скрытом слое требует совсем не экспоненциальный рост, а линейный, хотя и существенно больший чем этого требует алгоритм backpropagation (см. таблицу 2). Но мы должны, четко понимать базовую разницу между этими подходами:

1. MLP-backpropagation использует нелинейную функцию активации, а перцептрон, тоже нелинейную, но пороговую, для которой нельзя посчитать градиент
2. В то время как Backpropagation — это глобальный алгоритм на основе градиента от общей ошибки, правило обучения перцептрона локально и использует лишь индивидуальную ошибку каждого нейрона
3. MLP-backpropagation для обучения вычисляет градиент и распространяет его назад, а перцептрон, не обучает слой S-A (в нем происходит “эффект случайности”, когда входной вектор отображается на пространство признаков в скрытом слое), и по правилу Хебба обучает только последний A-R слой
4. Для MLP-backpropagation используется промышленная реализация с использованием библиотеки TorchSharp с обучением пакетами = 32, т.е.

максимально оптимизированно. А для перцептрона используется “наивная реализация” на С# без какой-либо оптимизации скорости.

Таблица 2.

Задача четность MLP-backpropagation (TorchSharp, MSELoss, Adam, batch_size = 32, shuffle = false, lr = 0.0001)

Кол-во бит	Кол-во возможных комбинаций	Число нейронов в скрытом слое	Итераций	Время	Требуемая память
8	256	50	7124	25 сек.	435 Мв
10	1024	100	3919	44 сек.	451 Мв
12	4096	100	6041	4 мин.	456 Мв
14	16384	150	3153	11 мин.	462 Мв
16	65536	200	518	7 мин.	484 Мв
17	131072	300	185	6,5 мин	517 Мв
18	262144	300	132	9,5 мин	546 Мв
19	524288	300	103	15 мин.	629 Мв
20	1048576	300	149	42 мин.	757 Мв

На рис. 1 и рис. 2 мы видим процесс схождения во время обучения для MLP+backpropagation и перцептрона. В целом они похожи, и имеют вид затухающих колебаний, но природа этих колебаний разная. Для MLP+backpropagation мы практически не видим колебаний, а затухание происходит по плавной экспоненциальной кривой. Это связано с подбором коэффициента learning rate, и достаточным для обучения числа нейронов. Если бы MLP дать меньшее число нейронов или выбрать более высокую learning rate, например, = 0.001, мы также увидели бы затухания, но чаще застревание алгоритма схождения в последних 2-4 ошибках. Для перцептрона нет необходимости подбирать коэффициенты, его весовые коэффициенты изменяются кратно единицы. И колебания становятся хорошо видны (зубчатая линия на графике), что является признаком нехватки числа нейронов.

Но и в том и другом случае, мы хорошо видим, как 97% примеров достаточно быстро, примерно за 30-50 итераций правильно относятся к классу, и потом следует длинный “хвост” процесса схождения. Это стало так обыденно, что появился термин “переобучение”, которым часто объясняют этот “хвост” обучения. Причем так, что предлагают остановить обучение, что это якобы снижает способность сети к прогнозированию, обобщению в целом. Но суть этого явления совсем в другом, и связанна с неточностью вычислений на практике. Конечно, в некоторых случаях это может быть связано с зашумленными данными, но мы специально выбрали задачу четности, где в принципе не может быть шума. Поэтому это вопрос не шума, а процесса обучения. В следующих разделах, мы будем анализировать, как случайность помогает перцептрону решать задачу. Но уже сейчас обратим внимание, на то, что чтобы различить каждую из возможных комбинаций, а при 20 битах их больше 1 млн., нужно чтобы активность нейронов была различима для каждого разного стимула. И MLP здесь ничем не отличается от перцептрона, хотя и использует функцию активации ReLu вместо пороговой, но важно лишь то активен нейрон или нет.

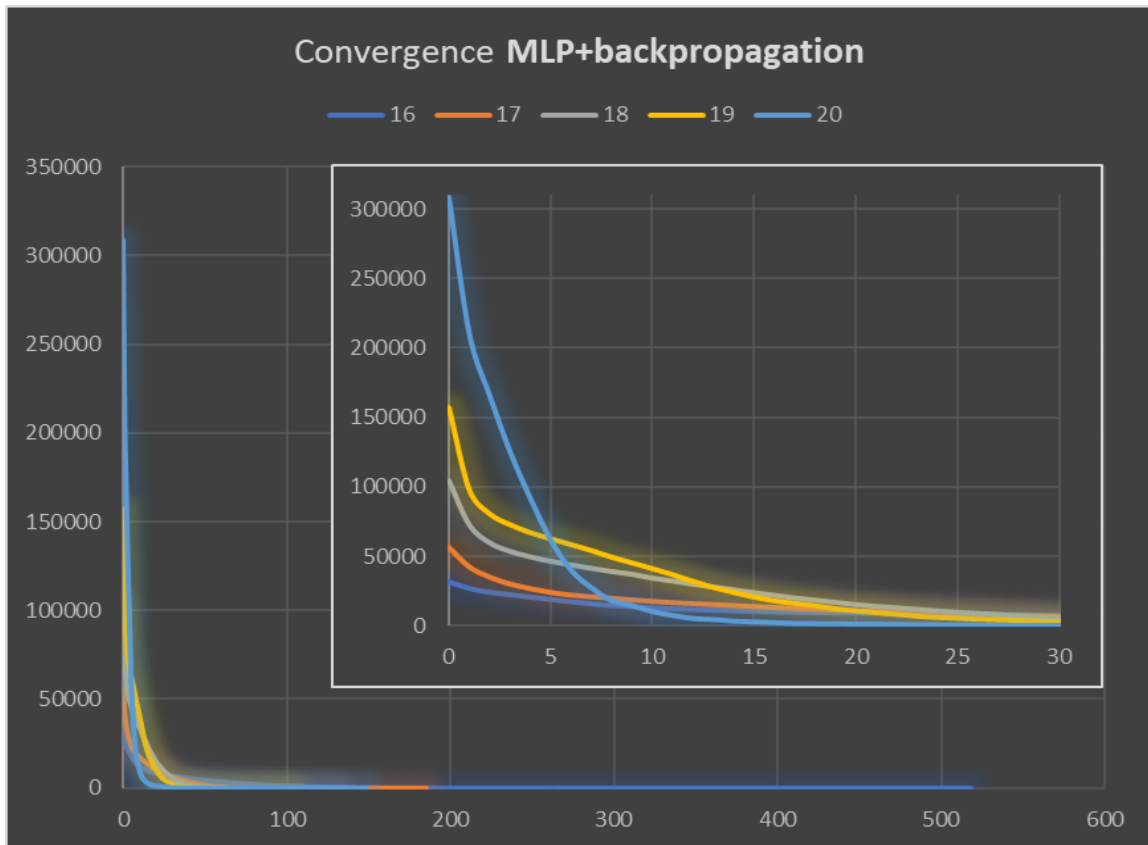


Рис. 1. Сходимость MLP + backpropagation

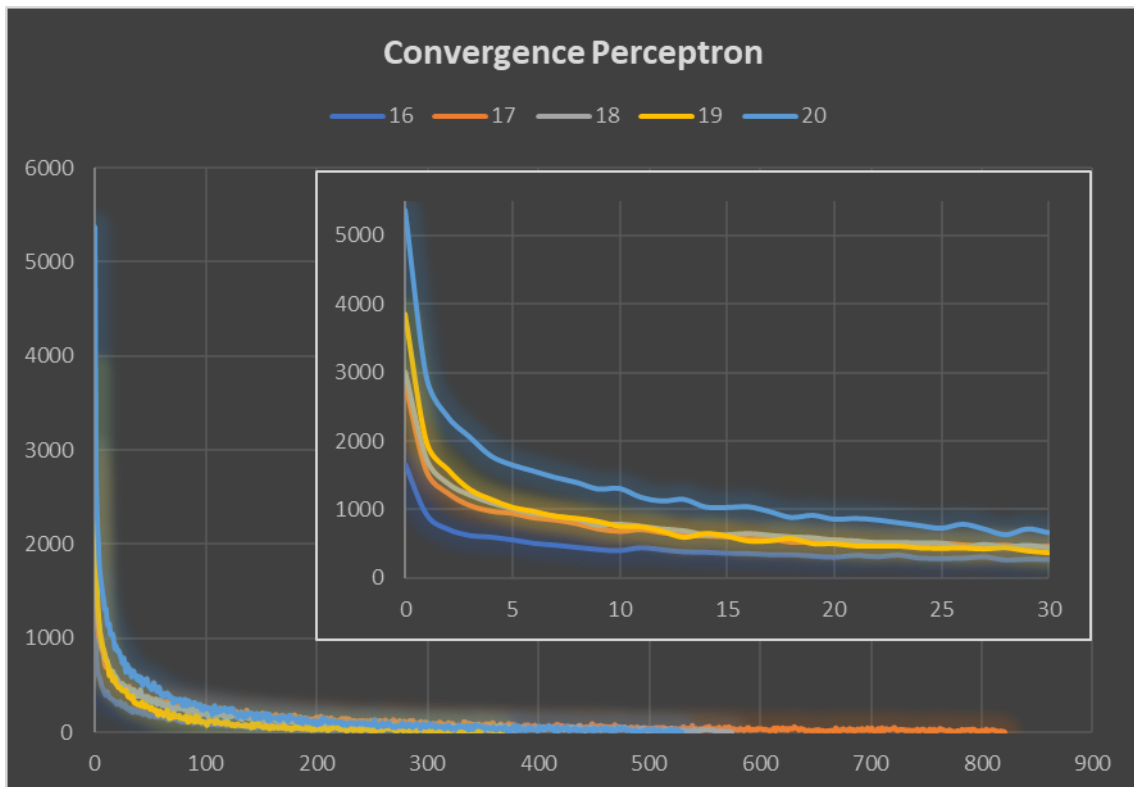


Рис. 2. Сходимость перцептрона

Для наглядности представьте следующий процесс: у вас есть 300 (20x15) пустых ячеек, и мы будем случайным образом, с равномерным распределением, бросать в них дротики. Сколько дротиков нужно кинуть, чтобы гарантированно заполнить все ячейки хотя бы одним дротиком. Сколько, дротиков будет кинуто напрасно? И главное, как замедлится процесс при заполнении доски дротиками, особенно когда будут оставаться пустыми последние 2-5 ячеек?

Эта задача так же известна, как задача о коллекционере (Coupon Collector's Problem) и она решается аппроксимацией выражения $E(T) = N * (1 + 1/2 + 1/3 + \dots + 1/N)$. В нашем случае, при $N = 300$, $E(T) = 1885$ дротиков. И ориентировочно 85% дротиков будут кинуты напрасно.

При этом процесс резко замедляется при заполнении последних ячеек. Среднее количество бросков – это математическое ожидание $= 1 / p$, где p – вероятность успеха. И чтобы попасть заполненную лишь на 50% нужно в среднем 2 броска, для 90% - 10 бросков, а для последней ячейки 300 бросков.

Именно такой процесс происходит внутри нейросети, но только еще более сложный. Изменение весов в S-A слое может снова “очистить доску”, и стимулы станут не различимы. А при увеличении числа нейронов, многие из них будут выполнять роль балласта, нужного лишь для того, чтобы удерживать вероятность различения стимулов на максимальном уровне. Особенно это важно для перцептрона, но и для MLP-backpropagation это влияет на схождение при подборе весовых коэффициентов, которые при недостаточной точности заставят “вибрировать” нейрон чередуя положение включен-выключен. Поэтому несмотря на то, что для backpropagation мы пытаемся точно высчитать градиент, но из-за округлений при вычислениях и влиянии всех нейронов на следующих итерациях, мы делаем неточные шаги приближения, и начинается дрожание, что и образует “хвост” при схождении.

Алгоритмы выбора случайности для S-A связей. Теперь нам остается рассмотреть, как именно формируются случайные связи в S-A слое. Часто это представляют как в алгоритме 2, в то время как наиболее эффективные связи получатся если они будут выбраны с помощью алгоритма 1.

Алгоритм 1: Двухфазная инициализация связей

Вход: SCount = n

Выход: Матрица весов WeightSA

для каждого A-элемента a:

 для j = 1 до 2n:

 сенсор s ← случайный(1, n)

 если j ≤ n:

 вес ← 1 // возбуждающая фаза

 иначе:

 вес ← -1 // тормозящая фаза

 WeightSA[s][a] ← вес

Алгоритм 2: Случайно-чередующая инициализация связей

Вход: SCount = n

Выход: Матрица весов WeightSA

для каждого A-элемента a:

 для j = 1 до 2n:

 сенсор ← случайный(1, n)

 тип ← случайный({-1, 1}) // равновероятный выбор

 WeightSA[сенсор][a] ← тип

Таблица 3.

Вероятности различных весов в S-A слое

	Алгоритм 1	Алгоритм 2
P(0)	13.5%	13.5%
P(1)	23.2%	43.3%
P(-1)	63.2%	43.3%
Энтропия (max = 1.585)	1.297 бит	1.436 бит

Из таблицы 3 мы видим, в алгоритме 1 меньше вероятность возбуждающих связей и меньшую энтропию. Последовательная фазовая инициализация создает аналог естественного отбора связей, где выживают только наиболее устойчивые возбуждающие соединения. Это приводит к формированию более контрастных и селективных паттернов активации, где редкие возбуждающие связи приобретают повышенную информационную значимость. В следующем разделе, мы будем отбирать более информативные признаки, и эти детали важны для реализации.

Итак, в целом, перцептрон и MLP+backpropagation обнаруживают очень сходные характеристики. Да, число нейронов для перцептрона требуется больше и главное скорость роста нейронов при усложнении задачи выше, чем у MLP+backpropagation. Но нейрон у MLP скажем так, более “тяжелый”, требует более сложных вычислений и занимает сходное для вычислений время. Но проблема в том, что дальнейший рост числа нейронов у перцептрона становится очень проблематичным. В алгоритме мы пользовались тем, что S-A веса фиксированные, и активность среднего слоя можно запомнить для каждого примера. Но уже в задаче четности с 20 битами, мы видим резкий скачек требуемой памяти. Если же отказаться от хранения активности среднего слоя, его придется пересчитывать при каждом показе нового примера. MLP+backpropagation оптимизирует это используя пакетное обучение и также существенный вклад делает оптимизатор Adam, для перцептрона же мы пока не использовали никаких оптимизаций, кроме сохранения активаций фиксированного скрытого слоя. Но как минимум, мы видим, что на старте MLP+backpropagation и перцептрон, как два бойца, находятся примерно в одной весовой категории, хотя и с достаточно разными характеристиками, из-за которых их достаточно сложно сравнивать.

2. Чистота это всё что вам нужно (Purity is all you need)

Одной из центральных проблем современного анализа нейронных сетей остаётся интерпретация скрытых представлений. Несмотря на высокую эффективность многослойных перцептронов (MLP) и их вариаций, внутренние активации скрытых слоёв часто рассматриваются как «чёрный ящик».

В данной работе мы выдвигаем тезис: главным свойством качественного скрытого представления является чистота окрестности (purity). Под чистотой мы понимаем локальную однородность классов в пространстве активаций: если ближайшие соседи активации принадлежат одному и тому же классу, то такое представление считается чистым. Мы показываем, что высокая чистота окрестности напрямую связана с возможностью последующего линейного разделения классов на выходном слое.

Для обеспечения высокой чистоты необходимо удалять «шумные» нейроны, которые смешивают классы в скрытом пространстве. В качестве метода отбора мы используем адаптированный алгоритм ID3: он позволяет анализировать бинарные активации нейронов и сохранять только те, чей вклад увеличивает информационную выгоду и, как следствие, чистоту. В отличие от классического применения ID3 в деревьях решений, в нашем подходе особое внимание уделяется построению деревьев по положительным и отрицательным активациям, что отражает специфику нейросетевой архитектуры.

Мы демонстрируем, что такой отбор не только радикально сокращает число нейронов (в 5–10 раз), но и автоматически приводит к формированию чистых локальных кластеров (средняя чистота 0.75–0.90). Это открывает путь к новой интерпретации скрытых слоёв: нейросеть должна учиться формировать представления с максимально высокой чистотой, и именно это свойство, а не конкретная архитектура или алгоритм обучения, является ключом к её успешной работе.

Метод ID3 (Iterative Dichotomiser), предложенный в [10], позволяет построить деревья решений на основании информационной выгоды. Этот способ в различных вариациях использовался, чтобы сократить число нейронов в MLP+backpropagation. Поэтому автор так же использовал этот метод в анализе активностей среднего слоя, что дало хорошие понимание – интерпретацию работы перцептрона.

Для применения метода ID3 к перцептрону, а точнее к анализу именно скрытого слоя нейросети, потребуется работать с битовыми векторами. Тогда имеется:

- Множество признаков (активности A-элементов): $A = \{a_1, a_2, \dots, a_n\}$
- Множество классов (требуемых реакций на R-элементе): $R = \{r_1, r_2, \dots, r_m\}$
- Обучающая выборка: $D = \{(x^{(i)}, y^{(i)})\}$, где $i = 1..N$, $x^{(i)} \in \{0,1\}^n$ - битовый вектор активации A-элементов для i-го примера из обучающей выборки; $y^{(i)} \in \{0,1\}^m$ - битовый вектор требуемых реакций

В качестве меры энтропии, будем использовать функцию:

$$H = -(p * \log_2(p) + n * \log_2(n)), \text{ где } p = \text{positives} / \text{total}, n = \text{negatives} / \text{total}.$$

Используемая автором адаптация алгоритма ID3 представляет собой рекурсивную процедуру с ограничением глубины рекурсии 100 уровней, позволяющую на основе построенного дерева решений классифицировать активность А-элементов с помощью правил вида if/else. Алгоритм последовательно отбирает наиболее информативные признаки, начиная с корневого узла с максимальной информационной выгодой и рекурсивно расширяя дерево вниз. В контексте нейросетевой архитектуры критически важным является построение двух деревьев: на основе наиболее информативного положительного признака и его инверсии - наиболее информативного отрицательного признака. В отличие от стандартной реализации, где второе дерево является логическим отрицанием первого, в нейросети невозможно использовать те же признаки для отрицательного ветвления, поскольку неактивные А-элементы не вносят вклад в весовые коэффициенты и не участвуют в обучении. Таким образом, для корректной работы перцептрона необходимо обеспечить отбор как положительных, так и отрицательных признаков при формировании правил классификации.

Мы так же можем разделить все множество А-элементов, на 2 и более части, и каждую часть анализировать отдельно. Это позволяет для небольших исходных признаков отбирать больше информативных признаков (см. Таблица 4 для 10 и 12 бит).

Несколько примеров, отобранных таким образом признаков в деревьях решений см. в приложении 1.

Таким образом, из изначально формируемых случайным образом S-A связей, рассчитанные активности А-элементов для каждого примера анализируются алгоритмом ID3, отбирая те нейроны в дерево решений, которые наиболее информативны, а остальные удаляются. После чего начинается обычное обучение перцептрона по правилу Хебба. Результаты такого обучения и на сколько возможно сократить число А-элементов см. в таблице 4. Разница как правило 5-10 кратная, но сам анализ занимает существенное время, поэтому такой подход важен именно для анализа, а не финального обучения.

Таблица 4.

Кол-во бит	Изначальное число нейронов	Число нейронов после сокращения	Итераций	Время на анализ ID3	Полное время
10	1600	70+56=126	689	0,6 сек.	1,2 сек.
12	1600	165+118=283	1786	3 сек.	11 сек.
14	3000	357	931	25 сек.	54 сек
16	5000	976	288	3,5 мин.	5 мин.
17	5000	1310	557	8,5 мин.	15 мин.
18	5000	? out of memory > 4Gb			

Чистота окрестности. Понятие кластерного анализа. Напомним его, пусть:

- $X = \{x_1, x_2, \dots, x_n\}$ — множество активаций (или признаковых векторов), $x_i \in R^d$.
- $y_i \in Y$ — метка класса, соответствующая x_i .

- $\rho(x_i, x_j)$ — метрика расстояния (будем использовать Хэммингово расстояние)
- Для точки x_i определим множество её k -ближайших соседей:
 $N_k(x_i) = \{x_{i1}, \dots, x_{ik}\}$, где расстояния упорядочены: $\rho(x_i, x_{i1}) \leq \dots \leq \rho(x_i, x_{ik})$.

Тогда **чистота окрестности** точки x_i определяется как:

$$P_k(x_i) = (1 / k) * \sum (\text{по } x_j \in N_k(x_i)) I(y_j = y_i),$$

где $I(\cdot)$ — индикаторная функция, равная 1, если условие выполняется, и 0 — иначе.

Если у нас 2 класса (четный и нечетный), то:

- $P_k(x_i) = 1.0 \rightarrow$ все k соседей принадлежат тому же классу, что и точка x_i . Абсолютно чистая окрестность.
- $P_k(x_i) = 0.0 \rightarrow$ все k соседей принадлежат другому классу. Максимально «грязная» окрестность.
- $P_k(x_i) = 0.5 \rightarrow$ соседи поровну делятся между двумя классами.

К сожалению, вычисление этого признака вычислительно затратно, поэтому мы смогли его посчитать только для задач малой размерности 10, 12, 14 бит (см. таблицу 4). Но мы видим, что это **надежный признак хорошей кластеризации** внутри сети, который имеет шанс на последующий этап линейной разделимости классов. Действительно, случайная генерация признаков и их последующий отбор с помощью алгоритма ID3 автоматически создает высокую чистоту окрестности от 0.75 до 0.90.

Это дает понимание того, какие именно признаки мы хотим формировать внутри нейросети, еще до линейного взвешивания они должны быть как можно чище разделимы, и наоборот, «грязных» нейронов быть не должно.

Таблица 5.

Кол-во бит	Изначальное число нейронов	Чистота окрестности min-avg-max	Активных нейронов (gain != 0)
10	1600	0.75 – 0.82 – 0.89	35
12	1600	0.78 – 0.89 – 0.94	75
14	3000	0.78 - 0.84 - 0.90	80
16	5000	? out of memory > 4Gb	

Gain. Еще одна важная оценка, которая вычисляется быстрее, но требует хранения активаций для всех примеров (что влечёт значительные затраты памяти) , — это количество нейронов, чей информационный вклад (gain) отличен от нуля (последний столбец таблицы 5).

Аналогично алгоритму ID3, можно определить важность активации нейрона в конкретном примере. Для каждого нейрона среднего слоя, алгоритм разделяет все примеры из обучающей выборки на два непересекающихся подмножества на основе бинарной активации: подмножество S_0 (примеры, где нейрон не активирован), подмножество S_1 (примеры, где нейрон активирован). Для каждого из этих подмножеств вычисляется энтропия, аналогично алгоритму ID3. В итоге мы

подсчитываем количество нейронов с ненулевой информационной значимостью (gain), что отражает реальное количество информативных нейронов в сети.

В процессе обучения информационная значимость (gain) разных нейронов будет изменяться во времени, напоминая осцилляторные процессы, похожие на синусоидальные функции. При этом линейный классификатор в последнем слое A-R постоянно отбирает те нейроны, которые будут более удобны и информативны для линейного разделения.

3. Двухуровневое и нормализованное обучение скрытого слоя перцептрона

В этом разделе мы опишем **Perceptron TL&NL (Perceptron with Two-Level & Normalization Learning)** и его алгоритм обучения. Описание разделим на три части: собственно описание алгоритма, анализ его характеристик и тестирование на задаче четность в сравнении с backpropagation и оригинальным перцептроном.

3.1. Разработка алгоритма обучения

Основы алгоритма заложил Розенблатт. Во-первых, он исходил из **правила локальной информации**: для любого A-элемента a_i значение ошибки $E_i(t)$ зависит только от информации, связанной с его активностью или поступающими на него сигналами, от веса его выходных связей и от распределения ошибки на его выходе в момент времени t .

Во-вторых, он сформулировал **правила обучения**, которые мы переформулировали для лучшего понимания:

1. Для каждого R-элемента, так же как и в простом методе коррекции ошибки, установим ошибку $E_r = R - r$, где R – требуемая, а r – достигнутая реакция
2. Для каждого A-элемента a_i ошибка E_i вычисляется, если ошибка E_r не нулевая, следующим образом:
 - a. Если элемент a_i активен и ошибка E_r по знаку отличается от веса v_{ir} , то с вероятностью p_1 коррекция равна -1
 - b. Если элемент a_i не активен и ошибка E_r по знаку совпадает с весом v_{ir} , то вероятностью p_2 коррекция равна +1
 - c. Если элемент a_i не активен, и ошибка E_r по знаку отличается от веса v_{ir} (или равна нулю), то вероятностью p_3 коррекция равна +1
3. Коррекцию ошибки E_i нужно делать для каждого активного элемента, предыдущего слоя.

Смысл этих правил в том, чтобы сделать неактивными те A-элементы, выходы которых увеличивают ошибку R-элемента, и наоборот сделать активными, те A-элементы, которые не активны, причем их выходы могли бы помочь скорректировать ошибку. Кроме того, с меньшей вероятностью p_3 учитывается случай, когда не реагирует ни один A-элемент и знак весов всех связей не правильный или равен нулю, то это приведет к возбуждению некоторых A-элементов. Так же это предотвращает ситуацию, когда A-элемент станет неактивным даже если во время обучения возникнет ситуация, когда все связи будут неправильными. Важно ещё отметить, что при обучении активность на входах S не учитывается.

И, в-третьих, у Розенблатта существовали, но достаточно невнятно разбросаны по тексту критически важные указания (выделение автора):

1. “сигнал от ошибки R-элемента используется для передачи сигнала коррекции обратно к сенсорному концу сети. Такая передача необходима, **если невозможно достаточно быстро осуществить коррекцию** на реагирующем конце сети”; что значит “невозможно” и “достаточно быстро”, остается неясным, но уже отсюда, становится понятно, что коррекция осуществляется не каждую итерацию.
2. “из экспериментов следует ... S-A связи стремятся к некоторой другой конфигурации, в то время как система [коррекция A-R слоя] продолжает пытаться приспособить веса своих связей к решению, чего она вполне может достигнуть при сохранении старой конфигурации. ... Для повышения устойчивости подкрепление для каждого стимула во всех экспериментах вводилось в A-R связи *до того*, как решался вопрос о том, следует ли вводить коррекцию обратно к S-A сети. Таким образом, **S-A связи изменялись только тогда, когда не удавались попытки скорректировать ошибку на уровне A-R связей.**”

Теперь ретроспективно оглядываясь назад, нужно признать, как мало оставалось сделать, чтобы такого рода алгоритм обучения смог бы заработать в полную мощь.

Понимая, из анализа алгоритма backpropagation, что элементы с порогом теряют информацию о величине активности отдельного нейрона, в результате чего в backpropagation стали использовать видоизмененную функцию порога ReLu (Rectified Linear Unit) $f(x) = \max(0, x)$. Важно отметить, что изначально backpropagation использовал другие функции активации сигмоид или тангенс. Но, ReLu по сути является пороговой функцией, но с пропуском активности.

С другой стороны, использование нормализации в нейронных сетях связано только с глубокими сетями (Ioffe & Szegedy, 2015). Дело в том, что, строго говоря нормализация обладает глобальной информацией, и противоречит правилу локальной информации Розенблатта. Это глобальное знание заключается в знании общего максимума среди всех активаций, т.е. вначале нужно получить все активации и только затем найти среди них максимум. Но знание максимальной активности нейронов во время показа стимула, вряд ли делает нейросеть менее биологически правдоподобной, скорее наоборот подобные механизмы глобального торможения и нормализации активности действительно существуют в биологических нейронных системах, где они реализуются через ассоциативные нейроны и механизмы латерального торможения. Поэтому использование этой информации скорее будет не нарушение правила локальной информации, а тонкий ключ к решению проблемы необходимости глобального знания в таких, по сути, последовательных задачах как четность и других, которые анализировал М. Минский.

Это, казалось бы, простая вещь – нормализация, может быть заменой вычисления производных от функции активации. Дело в том, что она так же несет информацию о том, в какой мере активность A-элемента превышает нужную по сравнению с общим уровнем активации в сети. При этом, следующий слой A-R линейного классификатора, может по-прежнему использовать пороговое значение 0 или 1.

Но это еще не все, нам оставалось “расшифровать” наследие Розенблатта спрятанное в методическом указании к проведенному им эксперименту: “*S-A связи изменялись только тогда, когда не удавались попытки скорректировать ошибку на уровне A-R связей*”. Что это значит, как это меняет алгоритм?

В результате этого у нас появился, то, что мы можем назвать алгоритм двухуровневого обучения:

Алгоритм 3: Двухуровневое обучение

ДЛЯ КАЖДОГО примера i :

```
AActivation(i); // Активируем A-элементы
RActivation(i); // Активируем R-элементы
bool e = GetError(i); // Есть ли ошибка?
if (e)
    // Первый уровень
    LearnedStimulAR(i); // Обучение на AR-слое
    RActivation(i); // Повторно активируем R-элементы
    bool e2 = GetError(i); // Ошибка исправлена?
    if (e2)
        // Второй уровень
        RandomChange(i); // Вероятностное правило коррекции SA
        LearnedStimulSA(i); // Нормализованное правило коррекции SA
        LearnedStimulAR(i); // Повторное обучение на AR – слое
        Error2++; // Кол-во ошибок неисправленных на втором уровне
    Error++; // Кол-во ошибок неисправленных на первом уровне
```

Общий вид такого алгоритма обучения, проходящего по нескольким уровням, серьезно отличается от backpropagation, но именно это позволяет ему работать по правилу “не чини то, что не сломалось”. И тут важно понять, что перцептрон обучается с двух сторон и со входов, и с выходов. Когда он обучается относить примеры, подаваемые на вход, с классами, требуемым на выходе – он обучает линейный классификатор на A-R слое. Когда же происходит наоборот, ему на выходе требуют класс, который должен относиться, к примеру на входе – он обучает линейный классификатор на A-S слое. Вместе эти процессы разнонаправленные, что и отмечал Розенблатт в цитате выше, и нужно дать одному из них приоритет, что автор и сделал в своем алгоритме. И становится, совсем не удивительно, что активность A-элементов начинает кластеризоваться, потому что эту активность вырабатывают обучением с двух сторон. Именно, поэтому важнейшим показателем успешности обучения в среднем слое является чистота окрестностей, о чем мы говорили в прошлом разделе.

Остается еще добавить, более мелкие, но все же значимые детали. Авторская переработка этого алгоритма, позволила в большой мере уйти от лишней стохастичности обучения. Вероятности p_1 и p_2 , благодаря коррекции на нормализованную величину, стало возможным убрать вовсе (принять всегда =1). На самом деле, вероятность коррекции у Розенблатта, и learning rate в алгоритме backpropagation играют одну и ту же роль: реже делать грубые изменения в обучении. Пока остается только вероятность p_3 , которой Розенблатт отводил вспомогательную

роль для инициализации связей значениями >0 , аналогично и backpropagation. Но главное, и тому, чтобы эти связи в процессе обучения снова же не стали $=0$, что для backpropagation менее вероятно, т.к. там используются числа с запятой, а не кратные единице. Мы можем рассмотреть алгоритм 4, в котором собраны вместе все виды обучения.

Алгоритмы 4: Виды обучения

RandomChange(стимул)

```
d = p3 (= 0.0001)
if OldError  $\neq$  0 then d = p3  $\times$  (OldError / HCount)
для КАЖДОГО r  $\in$  [0, RCount):
    для КАЖДОГО j  $\in$  [0, ACount):
        if AField[j]  $\leq$  0 TO
            для КАЖДОГО i  $\in$  [0, SCount):
                if random() < d
                    WeightSA[i][j] += c3 (= 0.01)
```

LearnedStimulSA(стимул)

```
для КАЖДОГО j  $\in$  [0, ACount):
    w = нулевой вектор длины SCount
    ЕСЛИ AField[j] > 0 TO // A –элемент активен
        для КАЖДОГО r  $\in$  [0, RCount):
            if E[r]  $\neq$  0 И sign(WeightAR[j][r])  $\neq$  sign(E[r]) TO
                для КАЖДОГО i  $\in$  [0, SCount):
                    w[i] -= AFieldNorm[j]
    ИНАЧЕ // когда A –элемент не активен
        для КАЖДОГО r  $\in$  [0, RCount):
            if sign(WeightAR[j][r]) = sign(E[r])
                для КАЖДОГО i  $\in$  [0, SCount):
                    w[i] += AFieldNorm[j]
    для КАЖДОГО i  $\in$  [0, SCount):
        WeightSA[i][j] += w[i]
```

LearnedStimulAR(стимул)

```
для КАЖДОГО j  $\in$  [0, RCount):
    для КАЖДОГО i  $\in$  [0, ACount):
        if AField[i] > 0
            WeightAR[i][j] += E[j]
```

Мы видим, что таким образом обучение AR слоя по правилу Хебба (LearnedStimulAR), остается полностью оригинальным. А обучение SA слоя разделяется на два: основное детерминированное, но с нормализацией (LearnedStimulSA) и второе стохастическое (RandomChange). Роль последнего критически важна – вопреки распространенному заблуждению, это не просто инициализация весов, а механизм поддержания постоянного шума в нейросети. Он необходим, чтобы алгоритм обучения мог

выбирать релевантные признаки, аналогично тому, как ветер раздувает пламя. Однако, при уменьшении ошибки требуется “прикрыть поддув” – эту роль играет выражение $p3 \times (\text{OldError} / \text{HCount})$, которое адаптивно корректирует исходную вероятность Розенблатта $p3$ в зависимости от количества оставшихся ошибок. Аналогичную роль, только более вычислительно сложно, с алгоритмом backpropagation проделывает алгоритм Adam, корректирующий learning rate. При этом, саму коррекцию мы делаем на небольшую, сравнимую с величиной нормализации, величину $c3$.

И наконец, для нашего алгоритма крайне важно, чтобы примеры на обучения перемешивались бы каждую итерацию, например, простым алгоритмом тасования Фишера — Йейтса.

3.2. Анализ обучения на задаче четность

Используя описанные в разделе 2 метрики, особенно чистоту окрестности, имеет смысл проследить не только финальное состояние нейросети, а ход её обучения. Позже мы уделим внимание “скоростным” характеристикам, они представлены в таблице 6, а здесь нас будет интересовать “качество” обучения. Дело в том, что текущая общепринятая методология, призывает оценивать обобщение нейросети на экспериментальной выборке, что называется, по факту. В данной статье, мы сознательно не делаем такие эксперименты, чтобы направить фокус изложения на других аспектах алгоритма. Можно лишь, упомянуть, что перцептрон **TL&NL** проверялся на задаче MNIST, и в несколько измененных условиях эксперимента давал немногих худшие результаты 95,5% против 96,5% правильных ответов в сравнении с MLP+backprop. Вместо таких “точечных/частных” проверок, из которых нельзя судить о обученной нейросети в целом, а результат обучения лежит в области статистической погрешности, необходимо сосредоточиться на развитии и анализе метрик, которые позволят судить о обучении в целом.

На рис. 3. Мы можем наблюдать изменение числа активных нейронов (AN) (информативная значимость gain неравна нулю), среднее попарное расстояние по Хеммингу (H) между активностями нейрона в среднем слое, или чистоту окрестности. При удачно выбранном числе нейронов в среднем слое и константах $p3$ (вероятность случайности) и $c3$ (шаг коррекции при случайности) AN и H в начале обучения рывками стремительно растут и долгое время затем находятся на одинаковом уровне. AN показывает то число нейронов, которые нейросеть может использовать для обучения. В данном примере у нас 240 нейронов, уровень разделения сигнала которых порядка 200 при максимуме 240, и всего активных 160 из 240. Выбором констант $p3$ и $c3$ можно добиться значений ближе к максимуму, но легко получить просто случайный сигнал. Отличить случайный сигнал от информативного можно по чистоте окрестностей. И данный пример интересен тем, что средняя чистота окрестностей в процессе обучения меняла ориентацию выше 0.5 (базируется на отличии 1 на выходе) и ниже 0.5 (базируется на отличии 0 на выходе). На качество будущего обобщения сети указывает то на сколько широкие приделы чистоты. На это указывает средняя чистота, и минимум (в данном случае) или максимум. У нас эти приделы находятся в широком разбросе от 0,2 до 0,8, с изменением направления центрального полюса (среднего). Это хорошо позволяет сети разделить класс четных от класса нечетных.

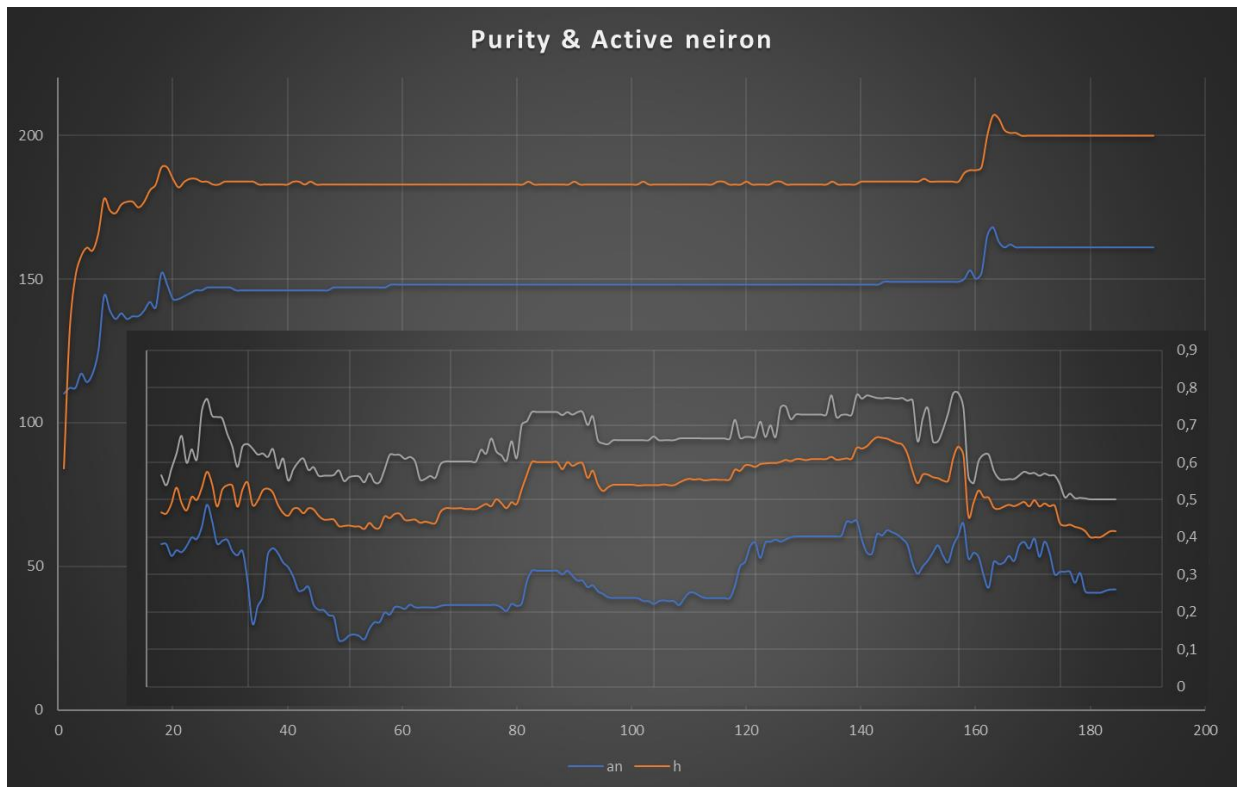


Рис.3. Чистота окрестности (минимум, среднее, максимум) и активность нейронов в среднем слое нейросети во время обучения. Задача четность 12 бит, 240 А-элементов, $p3 = 0.0002$, $c3 = 0.0001$

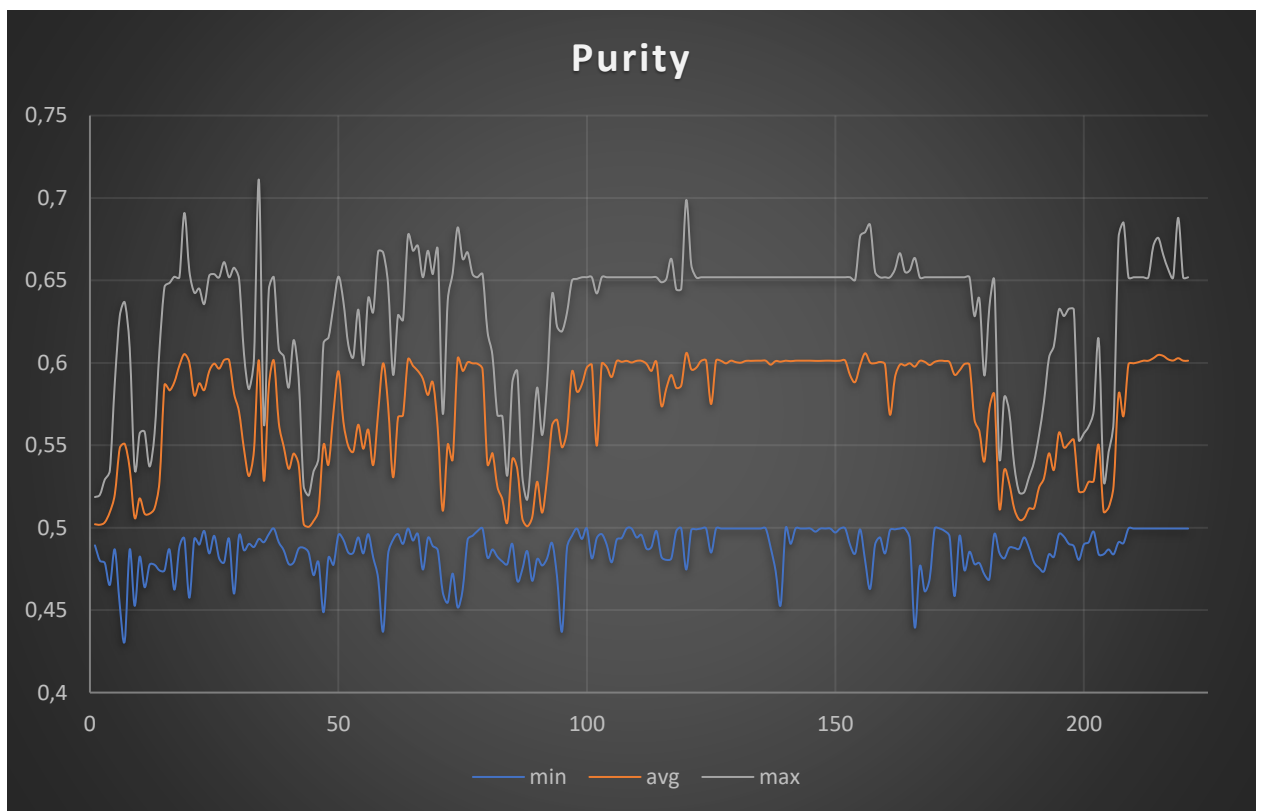


Рис.4. Чистота окрестности в среднем слое нейросети. Задача четность 12 бит, 300 А-элементов, $p3 = 0.0001$, $c3 = 0.01$.

Более стабильное поведение демонстрирует сеть при $p3 = 0.0001$, $c3 = 0.01$ (рис. 4). В этом случае чистота меняется в узком диапазоне 0.5 – 0.7, при среднем значении немногим больше 0,6. Это хорошо сказывается на обучающем процессе, он будет стабильно идти за разумное число итераций, но потенциально у сети не наступит возможность выбрать признаки, которые будут более коррелированные по классам.

Мы видели, что подбором констант можно с 300 нейронов в среднем слое сократить их количество до 240. Обучение будет более агрессивное в смысле использования фактора случайности, оно может быстрее обучиться за меньшие число итераций или с меньшим числом нейронов, но повышается фактор риска того, что не сойдется вовсе. Но используя такое “экстремальное” обучение можно добиться лучших результатов. Например, если оставить запас в необходимом числе нейронов в среднем слое =300, повысить вероятность случайности $p3=0.002$, но уменьшить силу влияния случайности с помощью $c3=0.00001$, то можно добиться практически мгновенного схождения. В данном случае за 30 итераций, хотя лучший результат был 16 итераций. Но вариант за 30 итераций интересен тем, что дал лучшие смещение чистоты окрестности (см. рис. 5). При этом нейросеть сразу использует число активных нейронов близко к максимуму (270 из 300), причем все они имеют почти максимальное расстояние по Хеммингу (275 из 300). Но главное, средняя чистота окрестности имеет высокое значение около 0,7, что ещё немногим ниже, чем при отборе нейронов с помощью алгоритма ID3 (см. табл. 5), но выше обычных значений. Наличие такой чистоты позволяет линейному классификатору быстро сойтись.

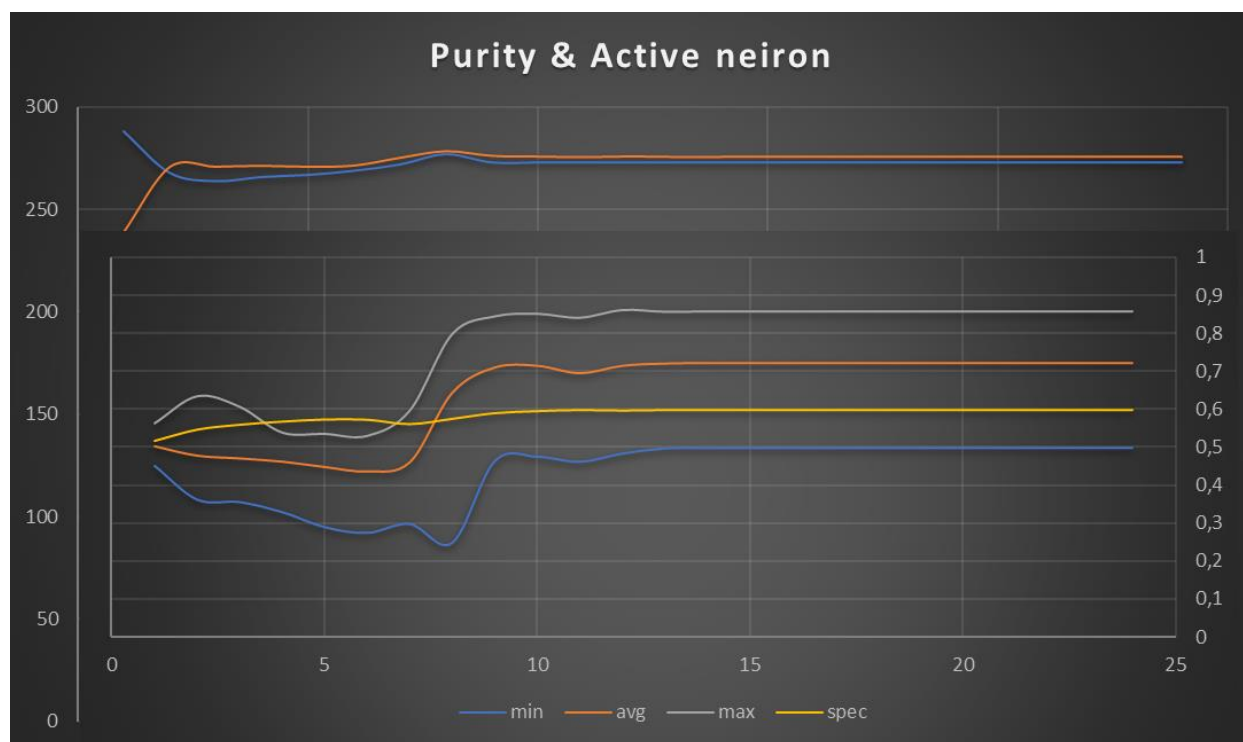


Рис.5. Чистота окрестности и прочие характеристики нейронной активности. Задача четность 12 бит, 300 А-элементов, $p3 = 0.002$, $c3 = 0.0001$.

Мы упоминали, что расчёт чистоты окрестностей вычислительно затратен, но вместо неё можно использовать упрощенную оценку (см. кривую spes на рис. 5). Она показывает специализацию нейронов на том или ином классе, и коррелирована с чистотой окрестностей.

3.3. Тестирование обучения на задаче четность

Как мы уже упоминали в первом разделе мы пользовались тем, что S-A веса фиксированные, и можно запомнить активность среднего слоя для каждого примера. Но когда средний слой нужно обучать, активность нейронов среднего слоя становится необходимым пересчитывать при показе каждого следующего стимула. Поэтому было необходимо решить вопрос с производительностью. Для этого небольшого участка, который в цикле, по сути, только прибавляет веса активных нейронов, была проведена модификация. Были использованы нативные команды процессора, т.н. AVX (Advanced Vector Extensions). Следует отметить, что TorchSharp в реализации backpropagation в том или ином виде их давно использует. Таким образом, мы улучшили производительность исключительно в том месте, где стало невозможным использовать “наивную реализацию” на C#. Дело в том, что излишняя оптимизация плохо влияет на читаемость кода и возможности его легкой модификации. Поэтому без крайней необходимости мы стараемся в разработке не использовать такие приемы. В следующем разделе, когда это станет важно мы увидим сравнение с полностью оптимизированным алгоритмом. Но алгоритм **TL&NL** для нашей задачи четности уже быстрее сходится, чем его классический конкурент backpropagation (сравните таблицу 6. и таблицу 2). Сравнивая же алгоритм **TL&NL** с классическим перцептроном, с другой стороны, мы можем заметить существенное уменьшение числа нейронов в скрытом слое. Конечно, это не то малое количество как у MLP+backpropagation, но и сами нейроны в **TL&NL** на порядок более простые.

Таблица 6

Кол-во бит	Число нейронов в скрытом слое	Итераций	Время без оптимизации	Время с оптимизацией Avx2	Активных нейронов (gain != 0)
8	100	1449	6 сек.	1 сек.	37
10	200	1035	15 сек.	2 сек.	78
12	270	345	90 сек.	2,4 сек.	135
14	300	75	138 сек.	3 сек.	210
16	500	71	17 мин.	30 сек.	384
17	700	93	30 мин.	2 мин.	620
18	900	42	48 мин.	2,5 мин.	?
19	1100	66	92 мин.	9,5 мин.	?
20	1300	57	?	17 мин.	?

4. Точность прогнозирования

В предыдущих разделах, мы стремились уменьшить число признаков (А - элементов), требуемых для решения задачи. И это понятно, т.к. обработка меньшего числа признаков требует меньше вычислительных затрат. Но выделяя только минимальное число признаков (и соответствующих А-элементов), и обучаясь только на части всех возможных примеров, мы рискуем построить слишком грубую модель. Её будет достаточно для решения задачи на обучающем множестве, но она будет плохо предсказывать. Представьте, что мы аппроксимируем окружность, и примеры нам показывают, что это многоугольник и во время прогнозирования мы исходим из того, на сколько углов мы обучили свою сеть. Но если правильно управлять этим процессом, то появится возможность предсказывать более плавно. Причем на том же самом количестве А-элементов, анализируя чистоту окрестностей, и выбирая такие константы как $p3/c3$.

Для анализа точности прогнозирования будем использовать классический тест MNIST по распознаванию рукописных цифр. Единственно, мы его несколько упростим, из 28x28 сетки, будем использовать 21x21 сетку, т.к. практически всегда крайние пиксели пусты. И вместо градации серого будем использовать черно-белое изображение.

Хотя задача MNIST структурно проще задачи четность, она требует больших размерностей, и поэтому имеет смысл для сравнения оптимизировать с помощью операций avx , все низко уровневые расчеты.

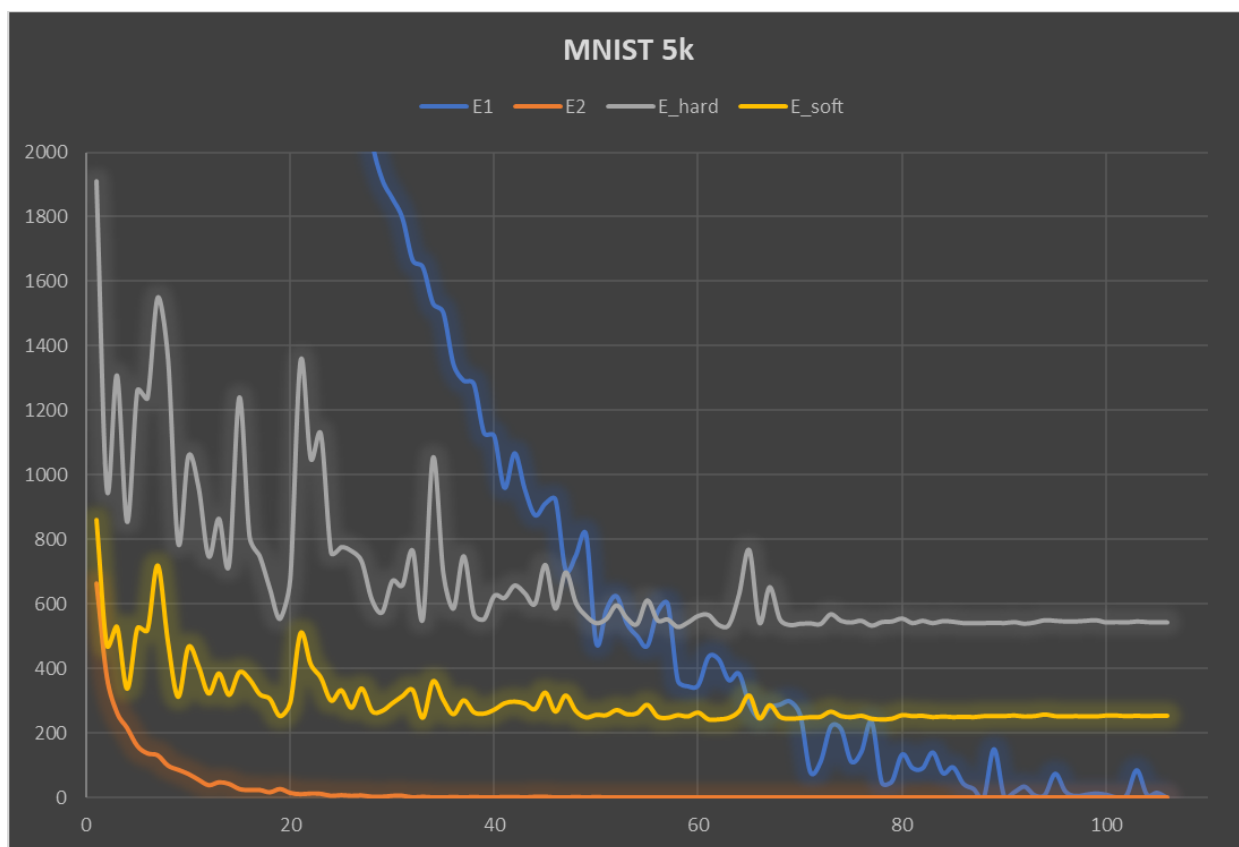


Рис.6.

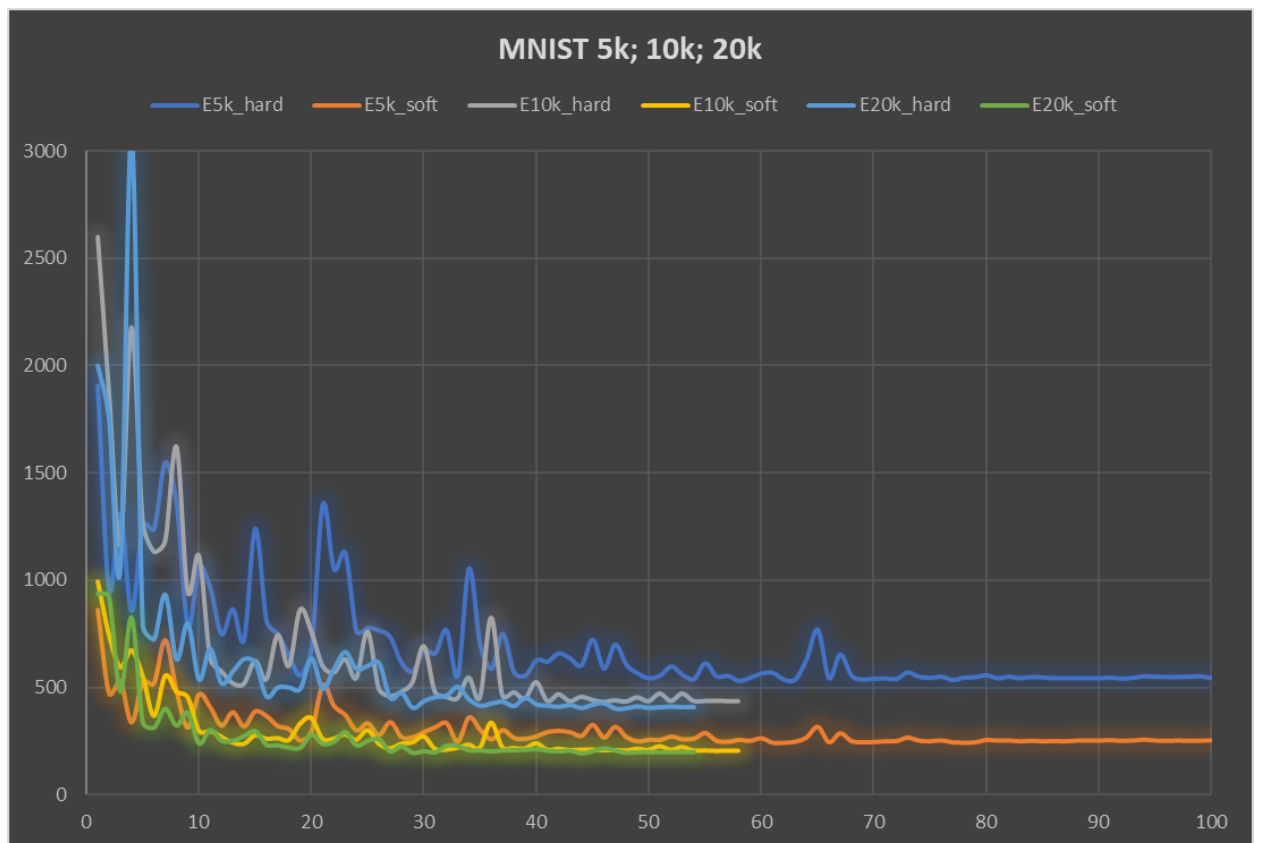


Рис.7.

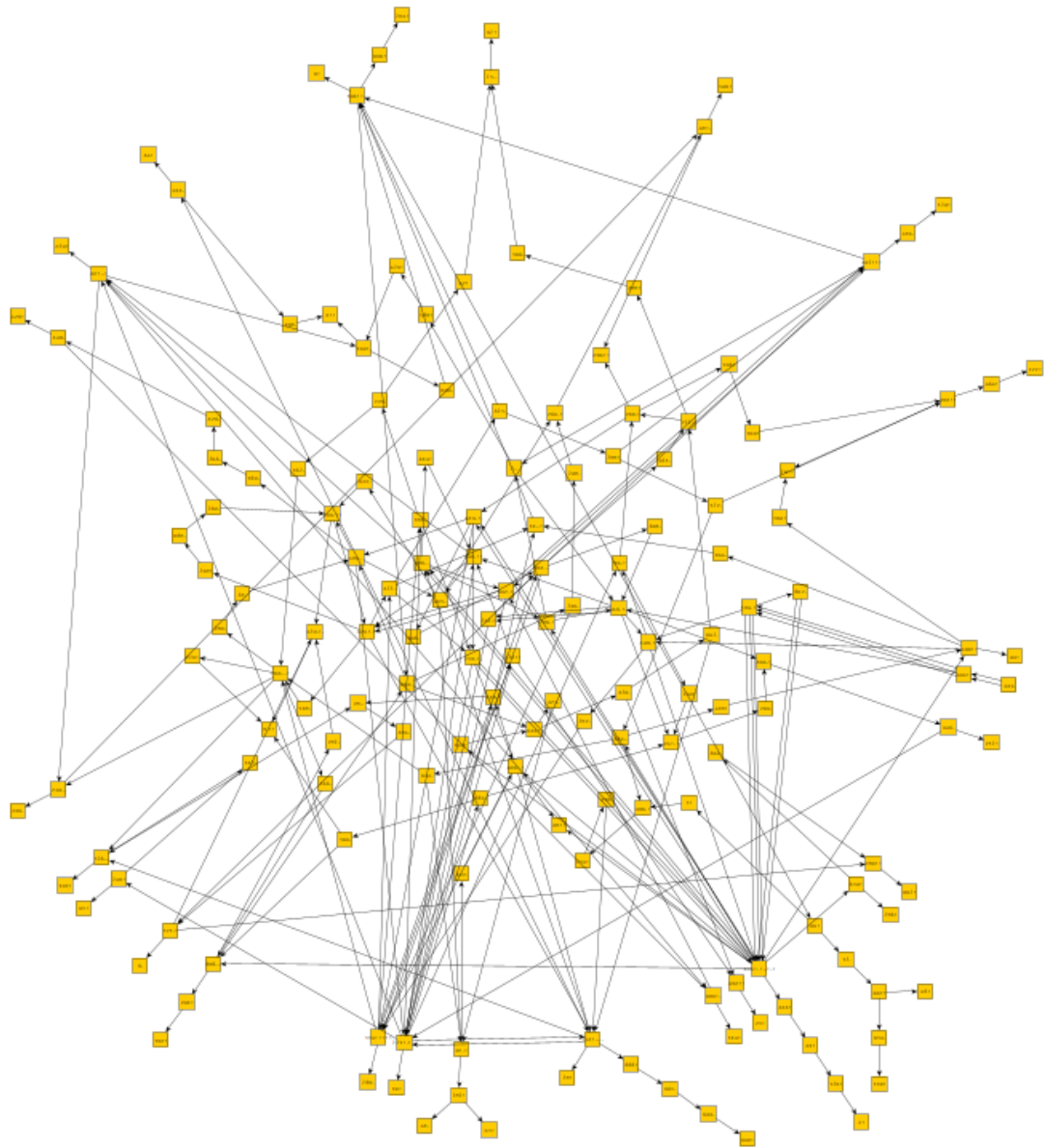
References

- [1] Rosenblatt, F. (1957). *The Perceptron—a perceiving and recognizing automaton*. Report 85-460-1. Cornell Aeronautical Laboratory
- [2] Rosenblatt, F. (1961). *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Report No. VG-II96-G-8. Cornell Aeronautical Laboratory
- [3] Minsky, M., Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. United Kingdom: MIT Press.
- [4] DARPA (1987) *Neural Network Study* Final Report. United States: The Laboratory.
- [5] Rumelhart, D. E., Hinton, G. E., Williams, R. J. (1985). *Learning Internal Representations by Error Propagation*. United States: Institute for Cognitive Science, University of California, San Diego.
- [6] Olazaran, M. (1971) *A Sociological History of the Neural Network Controversy*. Advances in Computers. United Kingdom: Academic Press.
- [7] Kirdin, A. N., Sidorova, S. V., & Zolotykh, N. Y. (2022). *Rosenblatt's first theorem and frugality of deep learning*. Entropy, 24(11), 1635
- [8] Kussul E., Baidyk T., Kasatkina L., Lukovich V., (2001) *Rosenblatt Perceptrons for Handwritten Digit Recognition*. — P. 1516—1520.
- [9] Yakovlev, S., (2009) *Perceptron architecture ensuring pattern description compactnes*, Scientific proceedings of Riga Technical University, RTU, Riga
- [10] Quinlan, J., (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann.

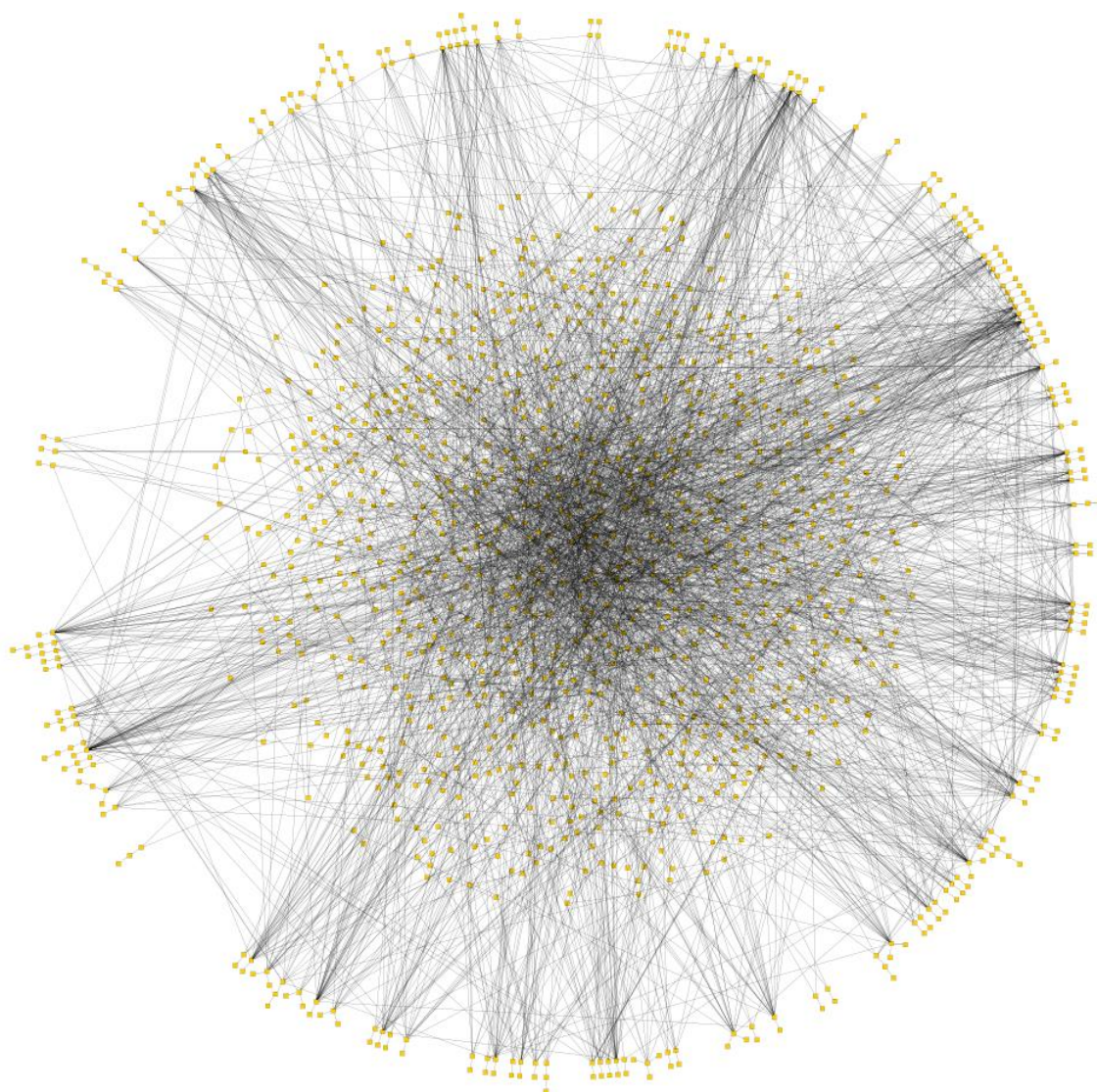
Sergey Ioffe, Christian Szegedy Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, 2015 <https://arxiv.org/pdf/1502.03167>

Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton, Layer Normalization, 2016, <https://arxiv.org/pdf/1607.06450>

Приложение №1.



Одно из двух деревьев, полученных методом ID3. Каждый узел – это отобранный нейрон (признак), который участвует в решении вопроса о принадлежности примера четному или нечетному классу в задаче четность с 12 битами. Из 1600 нейронов отобрано 283 всего, одно дерево состоит из 165, второе из 118. Эти два дерева не пересекаются. Чаще всего используемые нейроны внутри круга.



Дерево, полученное методом ID3. Каждый узел – это отобранный нейрон (признак), который участвует в решении вопроса о принадлежности примера четному или нечетному классу в задаче четность с 17 битами. Из 5000 нейронов отобрано 1310. Чаще всего используемые нейроны внутри круга.